

DRIVIX SMART PARKING

Comprehensive Frontend & Technology Stack Report

1. Executive Summary

Drivix is a high-fidelity, interactive smart parking and digital mobility solution designed for urban drivers. The application solves key modern commuting pain points through a single dashboard integration. Major functionalities include real-time parking slot discovery, visual slot selection map interfaces, dynamic reservation locking, simulated cashless transaction checkouts, and downloadable digital verification tickets.

This report reviews the detailed front-to-back technology stack, directory organization, page layout structures, and recent architectural improvements implemented to enhance the user experience, layout responsiveness, and synchronization accuracy.

2. Frontend Architecture & Tooling Stack

The frontend is built upon a fast, modular React 19 single-page application (SPA) architecture, utilizing modern developer tooling and lightweight libraries for dynamic assets.

Core Dependency	Version	Operational Role
React 19	v19.2.4	Core rendering library utilizing hooks, component states, and context providers.
Vite	v8.0.1	Next-gen development bundler facilitating hot module replacement (HMR).
React Router DOM	v7.13.2	Client-side routing engine supporting layout nesting and Admin protection routes.
Socket.IO Client	v4.8.3	Enables persistent real-time socket connections for slot state synchronization.
Framer Motion	v12.38.0	Manages UI animations, micro-interactions, and transition layouts.
Firebase SDK	v12.11.0	Integrates phone-number based authentication and OTP handling.
HTML to Image	v1.11.13	Converts digital ticket DOM components into downloadable PNG receipts.
Lucide React	v1.6.0	Provides vector-based modern iconography for clean aesthetics.

3. Backend Infrastructure Stack

The backend functions as a Node.js REST API and real-time Socket.IO server connected to a cloud MongoDB Atlas instance.

Service / Tool	Version	Functionality & Scope
Express	v4.19.2	Handles REST middleware routing for profiles, bookings, and slots.
Mongoose	v9.7.4	MongoDB object modeling tool defining schemas for bookings, locations, and users.
Socket.IO	v4.8.3	Manages dynamic namespaces, channels, and client status event emissions.
Helmet	v8.2.0	Applies HTTP headers for CORS policies, Content-Security-Policy, and XSS guards.
jsonwebtoken (JWT)	v9.0.3	Decrypts and validates authorization headers for secure client requests.
BcryptJS	v3.0.3	Performs heavy-duty salted hashing of login passwords on user registration.
Morgan & Nodemon	v1.10 & v3.1	Morgan logs terminal requests; Nodemon triggers live reloads on dev server.

4. Key Component & Page Registries

Navbar.jsx

Constructs the dynamic visual navigation bar. Employs light/dark theme toggles utilizing custom video assets (`sun.webm`) and vanilla CSS variable attributes. Includes a hamburger-toggle animation drawer overlay that delivers profile shortcuts, admin panels, and logout options directly to mobile devices.

HeroSection.jsx

Manages the landing page's main promotional interface. Displays live statistics fetched from the server. Features an interactive three-way selector tool supporting local searches for Parking Locations, and live simulated queries for FASTag Wallet recharges and unpaid E-Challan tickets.

SlotLayout.jsx

Renders the interactive graphical map grid representing active parking slots. Utilizes color-coded states to distinguish available slots from booked ones, handles dynamic Socket.IO listeners to receive slot updates instantly, and starts a temporary 5-minute database reservation lock when a slot is tapped.

SlotBookingForm.jsx

Manages the booking checkout flow. Autofills user profile vehicles, calculates price/hour rates, triggers a simulated credit card payment gateway, and enforces phone-number OTP validation. Hosts a countdown alert matching the active slot reservation timer.

Ticket.jsx & Profile.jsx

`Ticket.jsx` draws a high-fidelity visual receipt containing booking details and a verification QR code. `Profile.jsx` allows users to manage multiple cars, track active parking sessions, and vacate slots.

5. System Design Patterns Implemented

A. Publish-Subscribe (Pub-Sub)

Used for real-time parking slot synchronization in the frontend. When a parking slot status changes in the database, the backend acts as a publisher, broadcasting a ``slotStatusUpdated`` event. Connected client web browsers subscribe to this channel and update their states instantly without continuous request-response cycles.

B. Distributed Lock (Lease Lock Model)

To prevent double-booking race conditions when multiple users select the same slot at once, a distributed lease lock is created in MongoDB (``status: 'temporarily_reserved'`` and ``reservationExpiresAt``). A periodic cron-like cleanup scheduler running on the backend server automatically releases expired leases.

C. Event-Driven Architecture (EDA)

State updates across bookings, session checkouts, and slot releases are modeled as asynchronous event dispatches. The client interfaces react to server event streams via WebSocket connections rather than synchronous static API loops, ensuring decoupled communication flows.

D. Health Checks (Uptime Monitoring)

A dedicated ``/api/v1/health`` API endpoint is exposed to query active database connectivity status, server uptime, and epoch timestamps. This allows container orchestration systems, cloud hosts (Vercel/Render), or external load balancers to monitor service health and trigger self-healing automatic restarts if the server goes offline.